

INVESTIGACIÓN

Concurrencia en C#: Tasks, Threads, Async/Await y Programación Paralela

1. Introducción

La concurrencia es uno de los pilares fundamentales de la programación moderna, permitiendo que las aplicaciones ejecuten múltiples operaciones de forma simultánea. En el contexto del desarrollo de software para sistemas bancarios y aplicaciones empresariales, la capacidad de manejar operaciones concurrentes de manera segura y eficiente es crítica.

C# ofrece mecanismos robustos para la programación concurrente a través de su Task Parallel Library (TPL), el modelo asíncrono con `async/await`, y colecciones thread-safe. Esta investigación examina en detalle cada uno de estos conceptos y su aplicación práctica en escenarios reales.

2. Tasks y Threads en C#

2.1 ¿Qué son los Threads?

Un Thread (hilo) es la unidad más pequeña de procesamiento que el sistema operativo puede ejecutar. Representa una secuencia de instrucciones que se ejecuta independientemente dentro del contexto de un proceso.

Características principales:

- Bajo nivel: Abstracciones cercanas al sistema operativo
- Costosos: Crear threads consume ~1MB de memoria
- Sincronización manual: Requieren lock, Monitor, etc.
- Complejidad: Propensa a deadlocks y race conditions

2.2 ¿Qué son las Tasks?

Una Task representa una operación asíncrona que se ejecutará en el futuro. Las Tasks son abstracciones de más alto nivel que los threads, proporcionadas por la Task Parallel Library (TPL).

Características principales:

- Abstracción de alto nivel: Abstractan detalles de gestión
- Reutilización: Usan ThreadPool para eficiencia
- Composición: Fácil combinación de operaciones
- Escalabilidad: Miles de tareas simultáneamente

- Manejo de excepciones: Propagación natural

2.3 Comparativa Task vs Thread

Aspecto	Thread	Task
Abstracción	Bajo nivel	Alto nivel
Creación	~1MB por thread	Muy económica
Escalabilidad	~1000 threads	Miles de Tasks

3. Uso de Tasks

3.1 Creación Básica

Ejemplo: Creación de Tasks

```
// Método 1: Task.RunTask task = Task.Run(() => {  
Console.WriteLine("Ejecutando");}); // Método 2: Con constructor y StartTask task2  
= new Task(() => { Console.WriteLine("Ejecutando");}); task2.Start(); // Método  
3: Con retorno de valorTask<int> taskWithResult = Task.Run(() => 42); int resultado  
= taskWithResult.Result;
```

3.2 Esperar Tareas

Ejemplo: Métodos de sincronización

```
// Esperar una tareatask.Wait(); // Esperar múltiplesTask.WaitAll(task1, task2,  
task3); // Primera en completarseTask first = Task.WhenAny(task1, task2); // Con  
async/await (preferido)await Task.WhenAll(task1, task2, task3);
```

4. Palabras Reservadas async y await

4.1 Conceptos Fundamentales

async y await proporcionan sintaxis legible para operaciones asíncronas. El compilador transforma el código en máquina de estados, permitiendo que await pause sin bloquear el thread.

Ejemplo: async/await básico

```
public async Task ObtenerDatos(){ Console.WriteLine("Iniciando..."); //  
await pausa sin bloquear thread await Task.Delay(2000);  
Console.WriteLine("Completado");} public async Task<int> ObtenerResultado(){  
await Task.Delay(1000); return 42;} // Llamarpublic static async Task Main(){  
await ObtenerDatos(); int resultado = await ObtenerResultado();  
Console.WriteLine(resultado);}
```

4.2 Manejo de Excepciones

Ejemplo: try/catch con async

```
public async Task ObtenerDatosSeguro(){    try    {        await LlamarAPI();    }
catch (HttpRequestException ex)    {        Console.WriteLine("Error: " +
ex.Message);    }    finally    {        Console.WriteLine("Limpieza");    }}
```

5. Paralelización de Tareas

5.1 Task Parallel Library (TPL)

Ejemplo: Parallel.For

```
// Procesamiento paraleloParallel.For(0, 100, i => {    Console.WriteLine(i);});//
Parallel.ForEach(int[] datos = { 1, 2, 3, 4, 5 };Parallel.ForEach(datos, item => {
Console.WriteLine(item);});// Con opcionesvar opciones = new ParallelOptions {
MaxDegreeOfParallelism = 4};Parallel.ForEach(datos, opciones, item => {
Console.WriteLine(item);});
```

5.2 Task.WhenAll

Ejemplo: Operaciones paralelas

```
public async Task ProcesoParalelo(){    var tarea1 = ObtenerDatoAsync(1);    var
tarea2 = ObtenerDatoAsync(2);    var tarea3 = ObtenerDatoAsync(3);    //
Ejecutar en paralelo    int[] resultados = await Task.WhenAll(        tarea1,
tarea2, tarea3    );    Console.WriteLine(string.Join(", ", resultados));}
```

6. Colecciones Concurrentes

C# proporciona colecciones thread-safe en System.Collections.Concurrent que sincronizan automáticamente el acceso:

Colección	Equivalente	Uso
ConcurrentBag	List	Agregar/quitar sin orden
ConcurrentQueue	Queue	FIFO
ConcurrentDictionary	Dictionary	Caché, búsquedas
BlockingCollection	IEnumerable	Productor/Consumidor

Ejemplo: ConcurrentDictionary

```
var cache = new ConcurrentDictionary<string, int>();// AddOrUpdate es
atómicocache.AddOrUpdate("contador", 1, (key, old) => old + 1);// TryAdd
segurocache.TryAdd("usuario", 100);// TryRemove segurocache.TryRemove("usuario",
out int valor);
```

7. Conclusiones

Esta investigación ha cubierto exhaustivamente los mecanismos de concurrencia en C#:

1. Tasks son superiores a Threads en código moderno
2. async/await proporciona sintaxis legible sin callbacks

3. TPL/PLINQ aprovecha multiprocesadores eficientemente
4. Colecciones concurrentes garantizan acceso thread-safe
5. La prueba de concepto integra todos los conceptos exitosamente

Recomendaciones para aplicaciones empresariales:

- Preferir async/await sobre Task.Wait()
- Usar colecciones concurrentes para datos compartidos
- Implementar error handling adecuado en async
- Considerar CancellationToken para operaciones largas
- Monitorear rendimiento en producción

El dominio de la concurrencia en C# es esencial para desarrollar aplicaciones escalables, seguras y eficientes, especialmente en contextos de transacciones de alto volumen.